

CryptoCopilot Demo Day: Presentation Speaker Transcript

1. Slide 1: The Core Mission (Introduction)

Hello everyone. I want to start with a situation that is common for many people. Imagine being new to the world of cryptocurrency. You look at a screen full of moving red and green candles, endless news alerts, and social media hype. It is overwhelming. A single trader usually ends up asking: "Given everything happening in the market right now, what should I actually think about these coins? If I made a trade, how would it perform?" I built **CryptoCopilot** to answer those questions. I designed it as a personal assistant that helps a trader make sense of the noise. But beyond the features, this is an engineering project first. My goal was to create a clean, production-grade system that demonstrates how to handle complex data pipelines and AI responsibly.

2. Slide 2: The Four Pillars of an Opinion

To give a helpful answer, the application cannot just look at one thing. It needs a balanced perspective. I designed the system to "think" using four different angles, or pillars. First, I use **Machine Learning** to estimate the probability of price movement over the next 24 hours. Second, I apply **Technical Analysis** using classic chart rules like RSI and Ichimoku. Third, the system checks **Fundamentals**, looking at the health of the blockchain. Finally, I use **News and AI Chat** to research the latest headlines. A component I call **"The Analyst"** then fuses these four views into one verdict. Unlike a black-box AI, the Analyst is **deterministic and auditable**. It follows clear rules to tell the user the direction, the conviction level, and whether the different signals actually agree.

3. Slide 3: A Polyglot Architecture

Engineering is about choosing the right tool for the job. That is why I built CryptoCopilot as a **"Polyglot"** system using multiple programming languages. The application runs in five Docker containers, split between **Python** and **Java**. I chose Python for data science and machine learning. I chose Java and Spring Boot for the backend "brain" because it is excellent for managing business logic. The most important design choice is that the database is the **only bridge** between them. There are no direct network calls between Java and Python. This makes the system robust; even if the Python service is switched off, the application still works perfectly using the last data it wrote.

4. Slide 4: The Data Layer

Any assistant is only as good as its information. CryptoCopilot focuses on a **"10-coin universe."** I chose these specific coins because they represent different mechanisms. For example, I included **Chainlink** as an oracle network and **XRP** for payments, making the project more technically interesting than just looking at ten identical coins. The system pulls from Binance, CoinGecko, and several RSS feeds. To date, the system has ingested exactly **231,082 rows** of data. I built the data layer to be resilient. If one source fails, the system logs the error and skips it. It never allows a single failed connection to crash the entire pipeline.

5. Slide 5: The Machine Learning Heart

The heart of the Python side is the machine learning pipeline. It turns raw price candles into "features," which are numbers that describe the market. I use a model called **XGBoost**. I chose this because crypto data is often noisy and features interact in non-linear ways—XGBoost is the best-in-class tool for this kind of tabular data. I also wanted to open the "black box." I used **SHAP values** to provide transparency. For every prediction, the interface shows the top three factors—like the day of the week or a specific momentum indicator—that pushed the prediction up or down. This explains the "why" behind the AI.

6. Slide 6: Honest Results and Calibration

I believe in total transparency. In crypto, many people promise 90% accuracy, which is usually a sign of an engineering error. CryptoCopilot is honest. The model has an **ROC-AUC score of 0.578**. This is a realistic, small edge over a random coin flip. In this field, a score much higher than 0.65 would actually be a **red flag for data leakage**, suggesting the model is "cheating" by seeing the future. I also implemented **Isotonic Calibration**. This ensures the probability numbers are "honest." If the system says there is a 70% chance of a price increase, that event should actually happen 70% of the time in the real world. This makes the confidence scores meaningful for the user.

7. Slide 7: The Backend Brain (Java & Spring Boot)

The application's brain is built with **Java and Spring Boot**. I structured it as a **"modular monolith"** rather than microservices. This keeps the code clean and organized without the extra complexity of a distributed system. One key detail is that the backend uses a library called **ta4j** to calculate its own technical indicators. I chose to recalculate these in Java rather than just reading them from the Python side. This prevents "data leakage" and ensures the backend is an independent thinker, not just a "slave" to the Python service.

8. Slide 8: The Researcher (AI & RAG)

For research, I use **RAG**, or Retrieval-Augmented Generation. Instead of letting an AI guess the news, the system fetches real facts from the database and asks the AI to summarize them with citations. The system uses **pgvector** to store text as numbers called "embeddings." I built a toggle to switch the chat between **Ollama** (local and private) and **OpenAI** (high quality). However, the embeddings always stay on the local Ollama model. This is a smart engineering choice: it avoids the **costly and slow re-indexing** of the entire database every time the user switches the chat provider.

9. Slide 9: The Analyst Aggregator

The Analyst is where the system reaches a final decision. It combines scores from all four pillars into a single conviction level. Because this logic is deterministic, the results are always consistent and easy to check. I also built a **"Hallucination Guard."** I do not allow the AI to invent numbers or prices. The system forces the AI to use only the real data from the database. If the AI tries to invent a number, or if the AI service is offline, the system defaults to a **plain, factual template**. The real data always comes before the "chat."

10. Slide 10: The Paper Trading Experience

Once the system provides an opinion, a user can act through **Paper Trading** . This uses fake money only—no real money, ever. To make it realistic, I built the engine to include a **0.1% taker fee** and **0.05% slippage** on every trade. The frontend is built with **React** and features professional candlestick charts. It includes a dashboard with metrics like the **Sharpe Ratio** and **Max Drawdown** . In my backtests, the strategy actually showed a **negative Sharpe ratio** . I report this honestly because a choppy market combined with trading fees can easily grind down a simple strategy. It proves the trading engine is working accurately.

11. Slide 11: Engineering First (Conclusion)

In conclusion, CryptoCopilot is about **engineering first** . I have built a system that uses three languages and five containers to create a complete, production-ready pipeline. The model's modest **0.375 F1 score** is a result of limited data—about two years of history—which is an honest engineering conclusion. While the market is hard to beat, the architecture here is solid and transparent. I have proven that I can build AI tools that are grounded in data and honest about their limits. Remember, this is a tool for **decision support** , not financial advice. Thank you for your time.